

PO Data Extraction — Optimised Architecture

Oct 6, 2026 · @Ketan Parmar

Summary

Turning off the model's thinking mode is the main fix: it should cut each PO from 2–3 minutes to well under a minute, with a target of under 30 seconds at 10 POs in parallel.

Today each PO goes to a self-hosted Qwen3.8-27B vision model on one H100. Each request takes 2–3 minutes. Qwen3.8 thinks before answering by default, at its deepest level (`xhigh`). For copying fields off a document, that reasoning adds thousands of hidden tokens and almost no value.

The plan has four parts:

- **Speed:** thinking off, the official FP8 model, built-in speculative decoding, a smaller JSON output, and header and line items extracted in two parallel calls.
- **Accuracy:** read the text layer of native PDFs, raise scan resolution to 150 dpi, and check every PO with maths, format and ERP rules.
- **Reliability:** a job queue with retries, timeouts and duplicate protection. Failed POs retry once with light thinking, then go to a small review queue.
- **Confidentiality:** no outbound internet from the GPU server; telemetry off in vLLM, Hugging Face and LiteLLM.

All latency figures in this document are estimates. Measure each change against the existing test set before moving to the next.

Current setup and requirements

Item	Today
Documents	Purchase orders as PDF, 2–4 pages (largest 5), no fixed format
PDF types	About 50% native, 50% scanned
Fields	50+ per PO, spread across pages, plus line items

Item	Today
Suppliers	About 30 suppliers send 70% of POs
Volume	About 100 POs per day; up to 10 at the same time
Model	Qwen/Qwen3.8-27B , full precision (BF16)
Serving	vLLM behind LiteLLM, guided JSON output
Hardware	1× H100 80 GB, 200 GB RAM, 16 vCPU; GPU used only for this model
Input	All pages of one PO as images at 120 dpi, in a single request
Latency now	2–3 minutes per PO
Latency target	Under 30 seconds per PO; up to 1 minute acceptable
Accuracy	Test set with known correct values exists
Review	No human review before data is used
Data rule	Confidential; must never leave the intranet

Root cause of the latency

The model is most likely spending most of each request on hidden reasoning before it writes any JSON.

Per the [model card](#), Qwen3.8 runs in thinking mode by default and writes its reasoning inside `<think>...</think>` first. Its `reasoning_effort` setting has three levels (`xhigh`, `medium`, `low`), and `xhigh` is the default. So unless thinking was switched off, every PO gets the deepest reasoning the model can do.

Why this costs so much time:

- The model writes output one token at a time. Reading input is fast; writing output is the slow part.
- A full JSON for 50 fields plus line items is roughly 1,500–3,000 tokens. Deep reasoning can add several thousand more.

- Copying values from a page is a reading task, not a reasoning task. Deep thinking adds time without adding much accuracy.

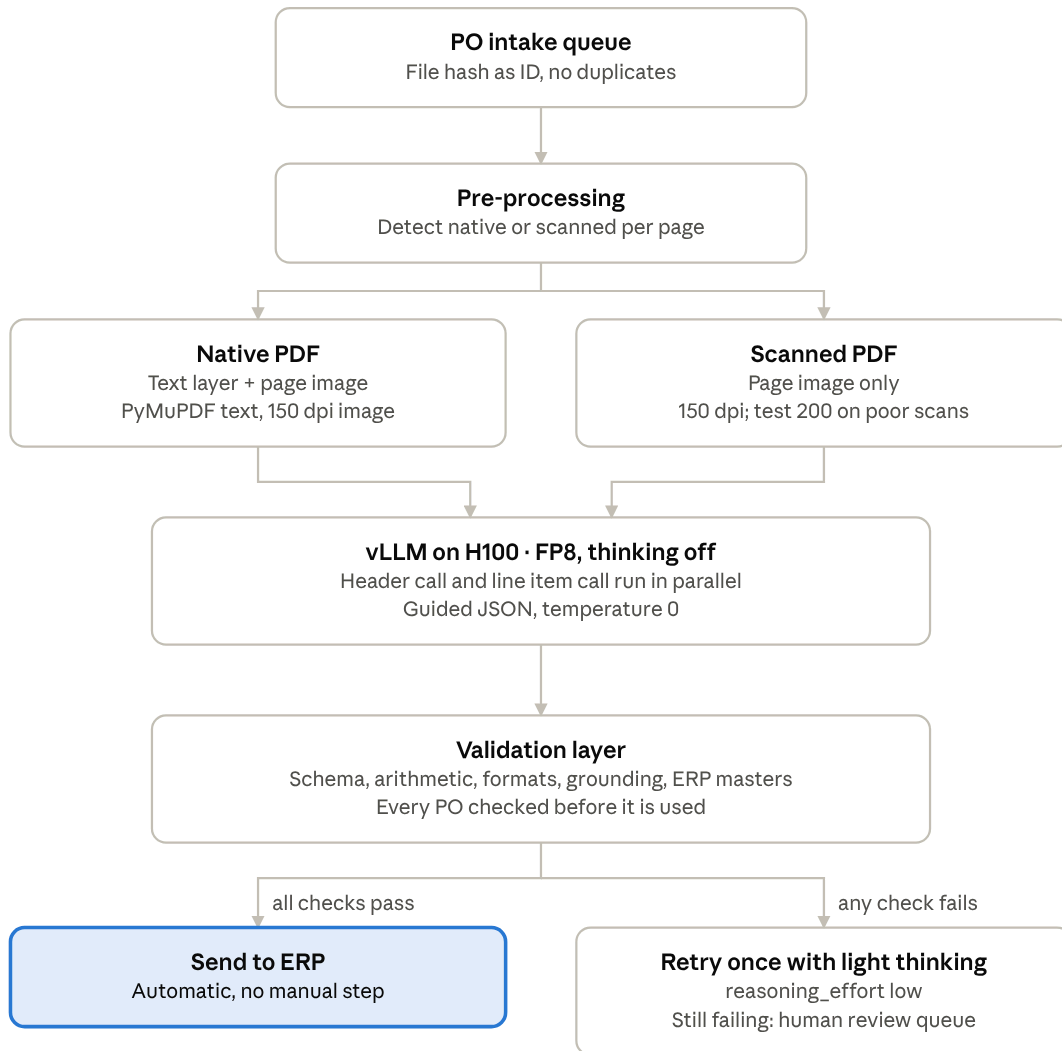
How to confirm: check `usage.completion_tokens` in a few current responses. If it is far bigger than the size of the JSON, thinking is the cause.

Other smaller causes: full precision (BF16) weights, a very large default context length, and vLLM settings not tuned for parallel requests. Each is addressed below.

Target architecture

Every PO passes through one queue, one model server and one set of checks; only POs that pass every check go straight to the ERP.

Only POs that pass every check reach the ERP



PO extraction flow · 6 stages, 2 outcomes

Native and scanned pages take different routes into the model, then merge. Failures get one automatic retry before a person sees them.

vLLM server configuration

Switch to the official FP8 model and launch vLLM with thinking off, a right-sized context, and speculative decoding.

The H100 runs FP8 natively. FP8 halves the model's memory (about 28 GB instead of about 54 GB), which speeds up token generation and leaves more room for parallel requests. The model also ships with a built-in draft head (MTP) that lets it predict several tokens

per step. JSON is very predictable, so this usually helps extraction. Settings follow the [vLLM recipe for Qwen3.8-27B](#).

```
export VLLM_NO_USAGE_STATS=1
export DO_NOT_TRACK=1
export HF_HUB_OFFLINE=1

vllm serve Qwen/Qwen3.8-27B-FP8 \
  --tensor-parallel-size 1 \
  --max-model-len 32768 \
  --max-num-seqs 16 \
  --max-num-batched-tokens 16384 \
  --gpu-memory-utilization 0.90 \
  --enable-prefix-caching \
  --limit-mm-per-prompt '{"image": 6, "video": 0}' \
  --reasoning-parser qwen3 \
  --default-chat-template-kwarg '{"enable_thinking": false}' \
  --speculative-config '{"method": "mtp", "num_speculative_tokens": 3}' \
  --generation-config vllm
```

Setting	What it does
Qwen3.8-27B-FP8	Official 8-bit model; smaller and faster on H100
--max-model-len 32768	Enough for a 5-page PO plus prompt and output; the 262K default wastes memory
--max-num-seqs 16	Up to 16 requests run together; covers 10 POs plus retries
--max-num-batched-tokens 16384	Lets image-heavy requests be read in fewer steps
--enable-prefix-caching	Reuses work when requests share the same start (see parallel calls)
--limit-mm-per-prompt	Caps images at 6 per request; blocks video
--reasoning-parser qwen3	Keeps any reasoning out of the JSON content if thinking is ever turned on
--default-chat-template-kwarg	Thinking off for every request unless a request turns it on

Setting	What it does
<code>--speculative-config</code>	Uses the built-in draft head to generate tokens faster
<code>--generation-config vllm</code>	Stops the model's default temperature 1.0 being applied silently
Environment variables	Stop vLLM and Hugging Face from calling home

The KV cache stays at full precision on purpose. Memory is not tight with these settings, so there is no reason to risk any accuracy loss there.

Checks after launch:

- Use a recent vLLM version (the recipe needs 0.17.0 or newer).
- Read `vllm:spec_decode_num_accepted_tokens_total` to confirm the draft head is actually helping.
- If speculative decoding clashes with guided JSON in your version, remove that one line. Everything else still applies.

Request settings for accuracy and consistency

Use temperature 0 and no repetition penalties, so the same PO always gives the same answer.

```
response = client.chat.completions.create(
    model="Qwen/Qwen3.8-27B-FP8",
    messages=messages,
    temperature=0,
    max_tokens=6000,
    extra_body={
        "chat_template_kwargs": {"enable_thinking": False},
        "presence_penalty": 0,
        "repetition_penalty": 1.0,
        "guided_json": PO_SCHEMA,
    },
)
```

- **Temperature 0:** the model picks its most likely answer every time. This gives repeatable output.

- **Presence penalty 0:** the model card suggests 1.5 for non-thinking mode. Do not use it here. It punishes repeated tokens, and POs repeat values on every line (same unit, same tax rate, same digits). It can quietly change correct values.
- **max_tokens 6,000:** a safety cap, not a target. Normal output should be far below it.

Token budget, roughly:

Part	Tokens
One page image at 150 dpi	about 1,500–2,500
Prompt and field definitions	about 2,000–3,000
5-page PO, total input	about 10,000–15,000
JSON output (50 fields + line items)	about 1,500–3,000

Input tokens are cheap: the H100 reads thousands per second. Output tokens are the slow part, so that is where to cut. These figures are estimates; read the real numbers from the `usage` field.

Reducing output size

Every output token costs time, so make the JSON as small as it can be without losing data.

- **Short keys:** use `po_no` instead of `purchase_order_number` in the schema. Map back to full names in code.
- **Nulls for missing fields:** no notes, explanations or "not found" text.
- **No extra whitespace:** turn off whitespace in vLLM's structured output settings (check the flag name for your vLLM version).
- **No calculated values:** don't ask the model for anything code can work out. Extract what is printed; compute the rest.
- **Skip pages with no fields:** if some POs carry terms and conditions pages with nothing to extract, don't send them (see open questions).

Native and scanned PDFs

Treat the two PDF types differently: native PDFs already hold exact text, so give that text to the model alongside the image.

How to tell them apart: read each page with PyMuPDF. If a page has a meaningful amount of text (for example, more than 50 characters), treat it as native. Otherwise treat it as scanned. A PO can mix both, so decide per page.

	Native PDF	Scanned PDF
Text source	PDF text layer via PyMuPDF (milliseconds, exact characters)	Model reads the image
Sent to model	Page image (for layout) + extracted text (for exact values)	Page image only
Resolution	150 dpi image	150 dpi (test 200 dpi on poor scans)
Main benefit	No misread digits; enables the grounding check	Small print such as GSTIN and part codes stays readable

Why raise from 120 to 150 dpi: at 120 dpi, small characters can blur and be misread. Higher resolution adds input tokens, but input is cheap. Compare 120, 150 and 200 dpi on the test set and keep the lowest one with no accuracy loss.

Parallel header and line item calls

Split each PO into two requests by field group and send them at the same time, so the PO takes as long as the slower call instead of both added together.

- **Call A — header:** the ~50 header fields (PO number, dates, supplier or customer, terms, tax details, totals). Sees all pages.
- **Call B — line items:** the line item table only. Sees all pages.

Split by field group, not by page, because fields are spread across pages.

Order the message for caching: put the page images (and text layer) first and the field instructions last. Both calls then start with the same content, and prefix caching lets the second call reuse the work of reading the pages.

Capacity: 10 POs at once becomes 20 requests. With `--max-num-seqs 16`, a few will briefly queue. That is fine at 100 POs per day. Raise it to 24 if the queue metric shows waiting.

Only add this step if POs are still over 30 seconds after the earlier changes. It adds some complexity in merging results.

Validation layer and exception handling

Only let a PO flow into the business process automatically when code-based checks prove its data is right; everything else goes to a short review queue.

Why this matters: with 50+ fields per PO, even 99.5% accuracy per field means roughly one PO in four has an error somewhere. Without human review today, nothing catches it. The goal is not to review every PO by hand, only the ones that fail a check.

Check type	What it checks	Catches
Schema	Every field has the right type; required fields are present	Missing or malformed values
Arithmetic	Qty × rate = line amount; lines add up to subtotal; subtotal + taxes = grand total	Most digit misreads
Format	Valid dates; GSTIN 15-character format and checksum; PAN format; numeric HSN codes; known currency codes	Misread IDs and codes
Grounding (native PDFs)	Every extracted value appears in the PDF text layer	Values the model invented or changed
ERP master data	Customer or supplier exists; item codes exist; PO number is not already in the system	Wrong party, wrong item, duplicates

What happens next:

1. All checks pass → send to ERP automatically.
2. Any check fails → retry that PO once with thinking on at `reasoning_effort: "low"`.
3. Still fails → send to a review queue with the failed fields highlighted. A person fixes only those fields.

Track the review queue size. It should be a handful of POs per day, and it is what protects the business process.

Supplier hints (phase 2)

About 30 suppliers send 70% of POs, so a few lines of supplier-specific guidance in the prompt can lift accuracy at almost no time cost.

- Identify the supplier first, from the text layer or a quick look at the header.
- Keep a small hints file per supplier, for example: "PO number is labelled 'Order Ref'; delivery date is in the footer."
- Add only that supplier's hints to the prompt. Unknown suppliers get the standard prompt.
- Write hints from real errors seen in the review queue, not guesses.

This is not rigid templating. The model still reads the whole PO; the hints only point it to the right labels.

Reliability, queueing and confidentiality

Put a job queue in front of the GPU so no PO is lost, processed twice, or stuck, and block all outbound internet from the GPU server.

Queue and workers:

- Use a simple job queue: Redis, RabbitMQ, or a Postgres table.
- Run a pool of async workers that render PDFs and call vLLM. The 16 vCPUs are plenty for PDF rendering.
- Give each job the file's hash as its ID, so the same PO is never processed twice.
- Set a timeout per request (for example, 120 seconds) and retry with backoff.
- Move jobs that keep failing to a failed-jobs list for someone to check.

LiteLLM: make sure its concurrency and timeout limits are not lower than vLLM's, or it will quietly queue requests. If it adds nothing beyond logging, consider calling vLLM directly.

Confidentiality:

- The firewall is the real guarantee: after downloading the model weights, block all outbound internet from the GPU server.
- Keep `VLLM_NO_USAGE_STATS=1`, `DO_NOT_TRACK=1` and `HF_HUB_OFFLINE=1` set.
- Turn off any telemetry or callbacks in LiteLLM.
- Keep PDFs, page images and JSON results on intranet storage only, with access limited to the service account and reviewers.

Monitoring

Log a few numbers per PO and watch vLLM's built-in metrics, so slowdowns and accuracy drops show up before users notice.

What	Where	Watch for
Input and output tokens per request	<code>usage</code> field, logged per PO	Output growing; thinking switched on by mistake
End-to-end time per PO	Worker logs	Above 30 seconds
Requests waiting	vLLM <code>/metrics</code>	Requests regularly queued
Draft acceptance	<code>vllm:spec_decode_num_accepted_tokens_total</code>	Low acceptance means the draft head isn't helping
Check failures by type	Validation layer	A rise after a prompt or model change
Review queue size	Review queue	More than a handful per day

Re-run the full test set after every prompt, model or setting change.

Rollout plan and expected results

Make one change at a time and run the test set after each, so the effect of every change on speed and accuracy is clear.

- ☐ Log `completion_tokens` on a few current requests to confirm thinking is the cause.
- ☐ Turn thinking off; set temperature 0 and presence penalty 0. Run the test set.
- ☐ Switch to the FP8 model with the new vLLM command. Run the test set.
- ☐ Shrink the output schema (short keys, no whitespace, no calculated fields).
- ☐ Add the native/scanned split and the 150 dpi change.
- ☐ Add the validation layer, retry path and review queue.

- ☐ Add the queue, workers and confidentiality settings.
- ☐ If still above 30 seconds, add the parallel header and line item calls.
- ☐ Phase 2: supplier hints for the top 30 suppliers.

Expected results (estimates, to be measured): thinking off plus FP8 should bring most POs to roughly 20–60 seconds, depending mainly on line item count. Speculative decoding and the parallel split should bring typical POs under 30 seconds with 10 running at once.

Open questions

- ☐ Is thinking currently on? Check `completion_tokens` and `reasoning_content` in a live response.
- ☐ What are the real input and output token counts per PO today?
- ☐ Are long POs long because of many line items (30+), or because of terms and conditions pages? If terms pages, they can be skipped.
- ☐ What is the largest number of line items seen on one PO?
- ☐ Which ERP system receives the data, and can the validation layer read its customer, supplier and item masters?
- ☐ Who will own the review queue for the few POs that fail checks?
- ☐ Which vLLM version is installed today?

Sources

- [Qwen/Qwen3.8-27B model card](#) — thinking mode default, `reasoning_effort` levels, sampling settings
- [vLLM recipe: Qwen3.8-27B](#) — FP8 checkpoint, MTP speculative decoding, reasoning parser, minimum vLLM version